

ΑΝΑΦΟΡΑ

Βασική λειτουργία

Ο κώδικας που μας ζητήθηκε να δημιουργήσουμε υλοποιεί ένα πολυνηματικό σύστημα καταγραφής και εκτέλεσης παραγγελιών ενός καταστήματος πώλησης πίτσας. Βασικός σκοπός είναι οργανωμένη και ταυτόχρονη εξυπηρέτηση πολλών πελατών, από την καταγραφή έως και την παράδοση των παραγγελιών τους. Έτσι η εκτέλεση της εργασίας στηρίζεται στην αξιοποίηση πολλαπλών νημάτων, όπου καθένα από αυτά θα απασχολείται από κάθε μια ξεχωριστή διαδικασία. Επομένως, τα δεδομένα και τα βήματα των ξεχωριστών παραγγελιών δεν θα συγχέονται ενώ εκτελούνται ταυτόχρονα, με αποτέλεσμα να επιτυγχάνεται η ταυτόχρονη υλοποίηση τους, όπως δηλαδή απαιτείται από μία πραγματική πιτσαρία.

Γενική δομή

Με βάση τις προϋποθέσεις της εκφώνησης ο κώδικας μας αποτελείται από δύο βασικά αρχεία:

- **p3190158-p3190196-pizza.c:** Περιέχει τον κορμό της υλοποίησης, συμπεριλαμβάνοντας τη κύρια συνάρτηση **main()** καθώς και τη συνάρτηση **order_thread()** που χειρίζεται κάθε παραγγελία.
- **p3190158-p3190196-pizza.h:** Ορίζει σταθερές, πόρους και εξωτερικές μεταβλητές που χρησιμοποιούνται για συγχρονισμό και καταγραφή στατιστικών.

Ροή εκτέλεσης προγράμματος

1. Στην κύρια συνάρτηση **main()** δημιουργούνται νήματα για κάθε παραγγελία, με τυχαία καθυστέρηση μεταξύ τους και στη συνέχεια κάθε νήμα εκτελεί τη συνάρτηση **order_thread()**
2. Εντός της **order_thread()** πραγματοποιούνται τα βήματα εκτέλεσης των παραγγελιών:
 - Δέσμευση τηλεφωνητή
 - Πληρωμή και πιθανή αποτυχία (συμβαίνει τυχαία με βάση συγκεκριμένη πιθανότητα)
 - Επιλογή τύπου πίτσας και κοστολόγηση (τυχαία επιλογή βάσει πιθανοτήτων για κάθε είδος)
 - Δέσμευση μάγειρα (προετοιμασία)
 - Δέσμευση φούρνου (ψήσιμο)
 - Δέσμευση συσκευαστή (πακετάρισμα)
 - Δέσμευση διανομέα (παράδοση)
 - Καταγραφή χρόνων (εξυπηρέτησης, κρυώματος)

Ο συγχρονισμός όλων των παραπάνω επιτυγχάνεται με τη χρήση **mutex** και **condition variables** για κάθε έναν πόρο ξεχωριστά. Το ίδιο συμβαίνει και για την τελική απογραφή, όπου γίνεται συλλογή και τύπωση στατιστικών όπως τα συνολικά έσοδα από τις παραγγελίες, τον αριθμό παραγγελιών (επιτυχημένων/αποτυχημένων) καθώς και τους χρόνους εξυπηρέτησης και παράδοσης.

Δυσκολίες

Όσων αφορά τους περιορισμούς που διαπιστώσαμε, παρά τις αλλαγές και τις δοκιμές που προσπαθήσαμε, παραμένουν να υφίστανται τρεις βασικοί:

- Η χρήση της μεθόδου `sleep()` κάνει ρεαλιστική προσομοίωση του χρόνου των παραγγελιών ώστε στην τελική απογραφή να εμφανίζονται δεδομένα που ανταποκρίνονται στην πραγματικότητα. Όμως αυτό συνεπάγεται και στην “ακριβή” πολυπλοκότητα χρόνου του προγράμματος, γεγονός που τον καθιστά ακατάλληλο για μεγαλύτερο όγκο παραγγελιών(δηλαδή αποτελεί σημαντικό περιορισμό στην δυνατότητα εξυπηρέτησης)
- Δεν υπάρχει οργάνωση στα νήματα οδηγώντας στην αδυναμία καθορισμού προτεραιοτήτων των παραγγελιών. Δηλαδή κάθε νήμα περιμένει τους διαθέσιμους πόρους και δεν επηρεάζεται από την πρόοδο των υπολοίπων παραγγελιών που τρέχουν ταυτόχρονα. Συνεπώς, ενώ σε ορισμένες περιπτώσεις το πρόγραμμα καταφέρναμε να εμφανίζει τις παραγγελίες με αύξουσα αριθμητική σειρά, στην επόμενη ακριβώς εκτέλεση με ίδια δεδομένα, οι παραγγελίες τυπώνονταν μπερδεμένες(ως προς το `id` της παραγγελίας)
- Οι χρόνοι εκπόνησης των διαδικασιών(παρασκευή, παράδοση) των παραγγελιών είναι διαρκώς αυξανόμενοι. Ενώ από την πρώτη έως και την τελευταία παραγγελία παρατηρούμε ρεαλιστικούς χρόνους, πιθανότατα λόγω των προηγούμενων ενεργειών, κάθε επόμενη παραγγελία παρατηρείται ελαφρώς πιο χρονοβόρα από τις προηγούμενες

Συμπέρασμα

Εν κατακλείδι, η εκπόνηση της συγκεκριμένης εργασίας ήταν απαιτητική ως προς το χρόνο αλλά και τις προκλήσεις που κληθήκαμε να αντιμετωπίσουμε. Παρόλα αυτά αποτελεί μια ρεαλιστική και άκρως εκπαιδευτική προσομοίωση ώστε να εξελίξουμε τις δεξιότητες μας σε λειτουργίες που εξυπηρετούν πολλούς χρήστες ταυτόχρονα, χαρακτηριστικό της πλειονηφίας των εφαρμογών που θα συναντήσουμε στο μέλλον, ειδικά σε εργασιακό περιβάλλον.